

Task2

Task2(a):

This query match 2 products from the same order, and we use the less than operator in the where section to filter duplicates pairs as {A,B} and {B,A} and self duplicates as {A,A} and {B,B}, then we return the two products names and the number of times they were ordered together, and sort them in descending order.

```
MATCH (p1:Product)<-[:CONTAINS]-(o:Order)-[:CONTAINS]->
(p2:Product)
WHERE p1.product_id < p2.product_id //To prevent duplicates
RETURN p1.product_name, p2.product_name, count(o) AS
nbrTimeTogether
ORDER BY nbrTimeTogether DESC
```

<pre> 1 MATCH (p1:Product)-[:CONTAINS]-(o:Order)-[:CONTAINS]->(p2:Product) 2 WHERE p1.product_id < p2.product_id //To prevent duplicates 3 RETURN p1.product_name, p2.product_name, count(o) AS nbrTimeTogether 4 ORDER BY nbrTimeTogether DESC </pre>			Table RAW Q O ↓
p1.product_name	p2.product_name	nbrTimeTogether	
1 "Nimbus Analysis"	"Orion Push"	3	
2 "GreenLeaf Even"	"Solace Personal"	3	
3 "Acme Figure"	"Orion Second"	3	
4 "Everest Deep"	"Zenith Happy"	3	
5 "Nimbus Nice"	"Orion Season"	3	
6 "Orion Network"	"Zenith Religious"	3	
7 "Orion View"	"Everest Fast"	2	
8 "Willow Sing"	"Everest Fast"	2	
9 "Solace Risk"	"Everest Fast"	2	
10 "NeoTech Property"	"Everest Line"	2	
			Fetch limit hit at 5,000 records. Started streaming after 128 ms and completed after 461 ms.

Task2(b):

In this query we want to calculate the support of each product. We start first by counting the total number of orders we have in the database. Then for each product we count the number of orders and we divide them by the total number of orders. (We multiply the number of orders by 1.0 in order to prevent integer result since both values are integers, so we will get a decimal fraction in the result instead of 0). Finally, we return the product name and the support in a list sorted by the support in a descending order.

MATCH (total_o:Order)

```

WITH COUNT(total_o) as totalOrders
MATCH (p:Product)<-[:CONTAINS]-(o:Order)
WITH p, totalOrders, count(o) as nmbrOrder
WITH p, totalOrders, nmbrOrder, (nmbrOrder * 1.0)/totalOrders as
productSupport
RETURN p.product_name, productSupport
ORDER BY productSupport DESC

```

Table RAW			
	p.product_name	productSupport	
1	"Pulse Money"	0.0018998100189981002	
2	"Everest Response"	0.0018498150184981502	
3	"GreenLeaf Rule"	0.0017998200179982	
4	"Zenith School"	0.0017498250174982502	
5	"Orion Range"	0.0017498250174982502	
6	"Zenith Stop"	0.0017498250174982502	
7	"GreenLeaf Card"	0.0017498250174982502	
8	"Everest She"	0.0017498250174982502	
9	"GreenLeaf Save"	0.0017498250174982502	
10	"Zenith Environmental"	0.0017498250174982502	

Started streaming 2,000 records after 183 ms and completed after 280 ms.

Task2(c):

In this query, we want to get all the products that were ordered together with a minimum support of 0.0002. We start again as we did

before in question (b) by getting the total orders. Then the same logic we did in question (a) to get two products without duplicates, then we count the support and filter for the ones that has a support superior or equal to 0.0002 and return them. The result is an empty list. We removed the condition and checked for the biggest support and it was "0.00014".

```
MATCH (total_o:Order)
WITH COUNT(total_o) as totalOrders
MATCH (p1:Product)<-[:CONTAINS]-(o:Order)-[:CONTAINS]->
(p2:Product)
WHERE p1.product_id < p2.product_id
WITH p1, p2, totalOrders, COUNT(o) AS sharedOrders
WITH p1, p2, totalOrders, sharedOrders, (sharedOrders *
1.0)/totalOrders AS sharedSupport
WHERE sharedSupport >= 0.0002
RETURN p1.product_name, p2.product_name, sharedSupport
```

```
1 MATCH (total_o:Order)
2 WITH COUNT(total_o) as totalOrders
3 MATCH (p1:Product)<-[:CONTAINS]-(o:Order)-[:CONTAINS]->(p2:Product)
4 WHERE p1.product_id < p2.product_id
5 WITH p1, p2, totalOrders, COUNT(o) AS sharedOrders
6 WITH p1, p2, totalOrders, sharedOrders, (sharedOrders * 1.0)/totalOrders AS sharedSupport
7 WHERE sharedSupport >= 0.0002
8 RETURN p1.product_name, p2.product_name, sharedSupport
```

No changes, no records

Completed after 228 ms

Task2(d):

In this task we need to calculate the confidence of every two products ordered together and get the ones with confidence above 0.15. We

start with counting the products p1 containing A, then we get the product p2 containing B. This time we replaced the less than operator with not equal to prevent just the pairs {A,A} while keeping {A,B} and {B,A}. Then, we calculate the confidence using the given formula and we return only the products fulfilling the confidence > 0.15 criteria.

```
MATCH (p1:Product)<-[:CONTAINS]-(o1:Order)
WITH p1, COUNT(o1) as totalA
MATCH (p1)<-[:CONTAINS]-(shared_o:Order)-[:CONTAINS]->
(p2:Product)
WHERE p1 <> p2
WITH p1, p2, totalA, COUNT(shared_o) AS sharedOrders
WITH p1, p2, (sharedOrders * 1.0)/totalA AS confidence
WHERE confidence > 0.15
RETURN p1.product_name, p2.product_name, confidence
ORDER BY confidence DESC
```

```

1 MATCH (p1:Product)-[:CONTAINS]-(o1:Order)
2 WITH p1, COUNT(o1) as totalA
3 MATCH (p1)-[:CONTAINS]-(shared_o:Order)-[:CONTAINS]-(p2:Product)
4 WHERE p1 <> p2
5 WITH p1, p2, totalA, COUNT(shared_o) AS sharedOrders
6 WITH p1, p2, (sharedOrders * 1.0)/totalA AS confidence
7 WHERE confidence > 0.15
8 RETURN p1.product_name, p2.product_name, confidence
9 ORDER BY confidence DESC

```

Table RAW

Q

⌵

⬇

	p1.product_name	p2.product_name	confidence
1	"Nimbus Nice"	"Orion Season"	0.1875
2	"Harbor Group"	"Orion Involve"	0.18181818181818182
3	"Zenith Religious"	"Orion Network"	0.16666666666666666
4	"Nimbus Should"	"Willow Local"	0.15384615384615385
5	"Pulse Take"	"Nimbus Us"	0.15384615384615385
6	"Harbor System"	"Solace Section"	0.15384615384615385
7	"Harbor System"	"Everest Region"	0.15384615384615385
8	"GreenLeaf Relate"	"Pulse Sea"	0.15384615384615385
9	"NeoTech Peace"	"Willow Military"	0.15384615384615385

Started streaming 9 records after 91 ms and completed after 204 ms.

Task2(e):

In this query we count the lift of each product pair and we return those positively correlated. We start by the total orders as we did before. Then, we get the total orders of A and the shared orders to count the confidence. After that we count the support of B, and we apply the formula to get the lift. Finally, we return the products with the lift > 1 ordered by it.

```

MATCH (total_o:Order)
WITH COUNT(total_o) AS totalOrders

```

```

MATCH (p1:Product)<-[:CONTAINS]-(o1:Order)
WITH p1, totalOrders, COUNT(o1) AS TotalA
MATCH (p1)<-[:CONTAINS]-(shared_o:Order)-[:CONTAINS]->
(p2:Product)
WHERE p1 <math>\Diamond</math> p2
WITH p1, p2, totalOrders, TotalA, COUNT(shared_o) AS
sharedOrders
MATCH (p2)<-[:CONTAINS]-(o2:Order)
WITH p1, p2, totalOrders, TotalA, sharedOrders, COUNT(o2) AS
TotalB
WITH p1, p2, totalOrders, TotalA, sharedOrders, TotalB,
(sharedOrders * 1.0) / TotalA AS confidenceAB, (TotalB * 1.0) /
totalOrders AS supportB
WITH p1, p2, totalOrders, TotalA, sharedOrders, TotalB,
confidenceAB, supportB, (confidenceAB * 1.0) / supportB AS lift
WHERE lift > 1
RETURN p1.product_name, p2.product_name, confidenceAB,
supportB, lift
ORDER BY lift DESC

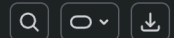
```

```

1 MATCH (total_o:Order)
2 WITH COUNT(total_o) AS totalOrders
3 MATCH (p1:Product)<-[:CONTAINS]-(o1:Order)
4 WITH p1, totalOrders, COUNT(o1) AS TotalA
5 MATCH (p1)<-[:CONTAINS]-(shared_o:Order)-[:CONTAINS]->(p2:Product)
6 WHERE p1 <=> p2
7 WITH p1, p2, totalOrders, TotalA, COUNT(shared_o) AS sharedOrders
8 MATCH (p2)<-[:CONTAINS]-(o2:Order)
9 WITH p1, p2, totalOrders, TotalA, sharedOrders, COUNT(o2) AS TotalB
10 WITH p1, p2, totalOrders, TotalA, sharedOrders, TotalB, (sharedOrders * 1.0) / TotalA AS
   confidenceAB, (TotalB * 1.0) / totalOrders AS supportB
11 WITH p1, p2, totalOrders, TotalA, sharedOrders, TotalB, confidenceAB, supportB, (confidenceAB *
   1.0) / supportB AS lift
12 WHERE lift > 1
13 RETURN p1.product_name, p2.product_name, confidenceAB, supportB, lift
14 ORDER BY lift DESC

```

Table RAW



	p1.product_name	p2.product_name	confidenceAB	supportB	lift
1	"Harbor Seek"	"Harbor Story"	0.14285714285714285	0.0008499150084991501	168.08403361344537
2	"Harbor Story"	"Harbor Seek"	0.11764705882352941	0.0006999300069993001	168.08403361344537
3	"NeoTech Into"	"NeoTech Better"	0.07142857142857142	0.00044995500449955	158.74603174603175
4	"NeoTech Better"	"NeoTech Into"	0.11111111111111111	0.0006999300069993001	158.74603174603172
5	"Harbor Group"	"Orion Involve"	0.18181818181818182	0.00114988501149885	158.1185770750988
6	"Orion Involve"	"Harbor Group"	0.08695652173913043	0.0005499450054994501	158.11857707509878
7	"Harbor Improve"	"Harbor Fire"	0.1	0.00064993500649935	153.8615384615385
8	"Harbor Fire"	"Harbor Improve"	0.07692307692307693	0.0004999500049995	153.8615384615385
9	"Zenith Study"	"Harbor Who"	0.09090909090909091	0.0005999400059994001	151.53030303030303
10	"Harbor Who"	"Zenith Study"	0.08333333333333333	0.0005499450054994501	151.53030303030303

Fetch limit hit at 5,000 records. Started streaming after 129 ms and completed after 1,050 ms.